Simulator 5: Claude Code for Continuous Integration

Alignment: Domains 3 and 4 · Runs on the Claude Agent SDK through your Claude login, against mock backends

Objective

Run the CI contract for pull request 418 of a stock-tracking service through the Agent SDK, with the same settings a real CI job passes to `claude -p`: `--output-format json` and `--json-schema`, explicit review criteria with four few-shot examples, one per-file pass per changed file followed by an integration pass, a re-run that suppresses prior findings, test proposals that respect the existing suite, a sync-versus-batch decision for three workloads, and a severity gate that fails the job.

Scenario (official exam scenario 5)

"You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives."

Primary domains: Claude Code Configuration & Workflows, Prompt Engineering & Structured Output.

Prerequisites

- `uv` and a Claude login (`claude auth login`). Run `uv sync` once in the `labs` folder. No API key.
- Open this simulator in your browser with `uv run serve.py` from `labs`, or run the commands below from this simulator's folder.

© 2026 Tertiary Infotech Academy · UEN 201200696W

What is in the folder

Path	Role
`repo/`	PR 418 applied: `stock/*.py` with an f-string SQL, an off-by-one loop, a widened `get_stock` signature and a stale caller; `tests/` with two existing tests; the `CLAUDE.md` the CI job loads
`prompts/claude-command.txt`	The three CI invocations with every flag explained
`prompts/review-criteria.txt`	Categorical report/skip rules, severity ladder, four few-shot examples
`finding.schema.json`, `test-proposals.schema.json`	Output contracts passed with `--json-schema`
`fixtures/changes.json`	Changed lines per file, existing test ids, one prior finding
`fixtures/workloads.json`	Pre-merge review, overnight debt report, weekly audit
`ci_review.py`, `verify.py`	Pipeline and acceptance checks
`hook_gate.py`, `repo/.claude/settings.json`, `repo/hooks/require-passing-tests.sh`, `repo/ci/`	Approval gate drill: PreToolUse command hook, mock approve and test scripts
`ci_subagents.py`, `cases/agents-review-team.json`	Agent SDK simulator: coordinator, three parallel reviewers, synthesis subagent
`cases/UC-*.json`, `casekit.py`	Eight practice-test use cases and the runner that executes and scores them
`tools/file_analysis.py`	Verbose-vs-compact analysis tool (UC-13)
`repo/stock/pricing.py`, `repo/.claude/commands/pr-check.md`	Noisy module for UC-39; slash command for UC-38

Guided tour

1. Read the command file. `-p` is there because CI has no keyboard; `--json-schema` makes the findings arrive already validated in `structured_output`; `--permission-mode dontAsk` with `--allowedTools Read Grep Glob` keeps the job read-only.

```
cat prompts/claude-command.txt
```

2. Read the criteria. Compare "report when behaviour contradicts the docstring or an adjacent test" with "be conservative", and read the two report examples and the two skip examples.

```
cat prompts/review-criteria.txt
```

3. Run the pipeline. Four per-file passes, one integration pass, one re-run with the prior finding in context and one test-generation call. Expect exit code 1: the gate blocks while the SQL injection and the off-by-one remain.

```
uv run ci_review.py; echo "exit $?"
```

© 2026 Tertiary Infotech Academy · UEN 201200696W

4. Open the per-file outputs. `repository.py:11` is critical/security with `detected_pattern` "f-string SQL"; `totals.py:10` is a high bug; the import and logging lines produce nothing.

```
uv run python -m json.tool evidence/review-repository.py.json; uv run python -m json.tool
    evidence/review-totals.py.json
```

5. Open the integration output. `api.get_stock` gained a `warehouse` parameter while `jobs.py:12` still passes one argument. No per-file pass can see this.

```
uv run python -m json.tool evidence/review-integration.json
```

6. Compare the first `repository.py` pass with the re-run. The prior finding was in the prompt, so the SQL line is not reported again.

```
uv run python -m json.tool evidence/review-repository.py.rerun.json
```

7. Check the test proposals and the API routing. The all-items total is already covered by `test_total_counts_all_items`, so it lands in `skipped_as_duplicate`; the blocking pre-merge check stays synchronous and the overnight and weekly jobs move to the Message Batches API.

```
uv run python -m json.tool evidence/review.json | grep -n -A12
    '"test_proposals"\|"api_routing"'
```

8. Run the verifier. Every output file is validated against `finding.schema.json`.

```
uv run verify.py
```

9. Stretch: fix the loop in `repo/stock/totals.py` and the caller in `repo/stock/jobs.py`, rerun, and watch the blocking count drop to one (the SQL line).

Subagents inside the pipeline: a local simulator

`ci_subagents.py` uses the Claude Agent SDK to run the orchestration behind practice question 2. A coordinator (your `LAB_MODEL`) spawns `security-reviewer`, `test-coverage-reviewer` and `maintainability-reviewer` in one response, each on `sonnet` with read-only tools, over the PR files in `repo/`. It then calls `review-synthesis`, a subagent with no tools and no memory of the others. The terminal prints a timeline as messages stream: elapsed time, agent name (from `SubagentStart`/`SubagentStop` hooks and `parent_tool_use_id`), the tool it used and how many findings came back. The count of `Agent` calls in a single assistant message is the proof of parallel spawning.

10. Run the correct design. The coordinator pastes every reviewer's complete JSON findings into the synthesis prompt; watch the synthesis prompt size in the final line.

```
uv run ci_subagents.py --pass-findings
```

11. Run the lossy design. The coordinator only asks synthesis to "summarise the review". Same reviewers, same findings, different handoff.

```
uv run ci_subagents.py --lossy
```

© 2026 Tertiary Infotech Academy · UEN 201200696W

12. Compare. The score is the share of file paths and failing-test names from the reviewers that survive into `evidence/pr-comment-<mode>.md`.

```
uv run ci_subagents.py --compare
```

Approval gate: a PreToolUse command hook, not a prompt

Practice question 3 (05:28): approvals must never post before the passing-test artifact exists. The rule is non-negotiable, so it lives in a hook, not in the system prompt. `repo/.claude/settings.json` registers `hooks/require-passing-tests.sh` for every Bash call. Claude Code pipes the hook JSON (`tool_input.command`, `cwd`) to the script on stdin. When the command is an approval and `ci/artifacts/tests-passed.json` is missing, the script exits 2 with the reason on stderr; Claude Code blocks the call and feeds that stderr to Claude. Exit 0 lets the call through. `ci/approve_pr.sh` is the mock approval (it appends to `ci/approvals.log`); `ci/run_tests.sh` is the mock test runner that writes the artifact.

13. Read the hook and the settings file, then run the two scenarios. Scenario 1 asks Claude to approve with no artifact: the command is denied before it runs and Claude quotes the hook's reason. Scenario 2 runs the tests first: the same prompt approves PR 418 and `approvals.log` gains one line.

```
cat repo/.claude/settings.json repo/hooks/require-passing-tests.sh
uv run hook_gate.py
```

14. Run the contrasts (two more calls) or read them in `evidence/hook_gate.json`. With only a PostToolUse hook the approval line is already in `approvals.log` when the hook complains. With a UserPromptSubmit reminder the policy reaches Claude as text; whether the approval posts is the model's choice, not a block.

```
uv run hook_gate.py --contrast
uv run python -m json.tool evidence/hook_gate.json | grep -n -B2 -A3 '"lesson"'
```

Labs that run on this simulator

Open this simulator in your browser: from the `labs` folder run `uv run serve.py`, then pick Simulator 5. In Claude Code, type `/lab` and the simulator number. A simulator is the system. A lab is one exam situation you run on it. Run any of these from the `labs` folder with `uv run run_lab.py <lab>`; each has its own PDF under its topic folder. An asterisk marks a lab run in class.

Lab	Task	Source	What it proves	Command
6*	1.3	Practice Q2	Same reviewers, same findings, and the PR comment loses them all	`uv run ci_subagents.py --pass-findings`
10*	1.4	Practice Q3	Approval must never post before the passing test artifact exists	`uv run hook_gate.py`
21*	2.3	Practice Q15	Review and test generation fight over one Claude call	`uv run casekit.py --case UC-15`
34	3.5	Practice Q14	Unknown requirements? Let Claude interview you first	`uv run casekit.py --case UC-14`

© 2026 Tertiary Infotech Academy · UEN 201200696W

Lab	Task	Source	What it proves	Command
36*	3.6	Practice Q18	The nightly job hangs because nobody can press Enter	`uv run casekit.py --case UC-18`
37	3.6	Practice Q38	A shared slash command still needs -p in CI	`uv run casekit.py --case UC-38`
38*	4.1	Practice Q39	"Only high confidence" didn't quiet the reviewer	`uv run casekit.py --case UC-39`
50*	4.6	Guide Q12	14 files, shallow comments: split the review	`uv run ci_review.py; echo "exit $?"`
52	5.1	Practice Q13	40-field tool results push the review to its limit	`uv run casekit.py --case UC-13`

Expected evidence

- `evidence/review-*.json`: one schema-valid file per changed file, the integration pass and the re-run.
- `evidence/review.json`: all findings, the suppressed prior, test proposals, API routing, the gate, per-call session ids and cost.
- `evidence/hook_gate.json`: both approval scenarios (denied reason, approvals.log, artifact) plus the two contrasts.
- `evidence/subagents-pass-findings.json`, `evidence/subagents-lossy.json`, `evidence/pr-comment-*.md`, `evidence/subagents-compare.json`: the simulator timelines, synthesis prompts, final comments and preservation scores.
- `evidence/cases/UC-<n>.json`: one scored file per use case.
- `evidence/verification.txt`: the twelve pipeline checks plus one PASS line per use case, with model, call count and cost.

Acceptance checks

- The pipeline and the documented commands use `-p`, `--output-format json` and `--json-schema`.
- Every finding carries `detected_pattern` so dismissed findings can be analysed later.
- Style-only and established-pattern lines never produce findings.
- A finding already in `prior_findings` is not repeated on the re-run.
- The cross-file arity mismatch is caught only by the integration pass.
- The gate exits 1 while critical or high findings remain.
- The PreToolUse command hook denies the approval (exit 2, reason quoted by Claude) until the passing artifact exists, then allows it.

Exam link

Task statements 3.6 (-p, --output-format json, --json-schema, prior findings, existing tests, CLAUDE.md standards), 1.4 and 1.5 (programmatic enforcement with a PreToolUse hook instead of prompt guidance), 4.1 (explicit criteria, severity ladder), 4.2 (few-shot examples), 4.4 (detected_pattern), 4.5 (sync vs Message Batches API), 4.6 (per-file plus integration passes), 1.6 (prompt chaining).

© 2026 Tertiary Infotech Academy · UEN 201200696W